

Chapter 27

Capstone: Your Analyst Portfolio

One question taken from the database to the memo, the check that turned a flattering finding into an honest one, and what happens to the work when a hiring manager opens it.

Draft chapter · v1

21 September 2026

Every query and every output was run on the full three-year database

Contents

Chapter 27. Capstone: Your Analyst Portfolio

Why this matters

In plain English

27.1 A question worth putting in a portfolio

27.2 SQL: the headline (a recap, not a re-teach)

27.3 Cleaning: the decisions, and whether they mattered

27.4 The check that changes the answer

27.5 Python: making the analysis arguable

27.6 The dashboard: one page, one decision

27.7 The memo

27.8 Selective reporting, and why a portfolio is where it starts

27.9 What a hiring manager does with your repository

27.10 Telling the story, in two minutes and in ten

27.11 What job-ready actually looks like

Common mistakes and how to spot them

In the real world: the finding Farah nearly led with

Tools

The project: build the portfolio

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 27. Capstone: Your Analyst Portfolio

Part II — The Analyst

Chapter at a glance

You will learn to: take one business question all the way from a database to a memo, using only what Part II taught · write down the cleaning decisions you made, and measure whether they changed the answer · find the check that turns a flattering result into an honest one, and report both · wrap the analysis in a function so it can be re-run and argued with · specify a one-page dashboard that serves a single decision · write the memo, including the part that says what you did not find · recognize selective reporting in your own portfolio, which is where it is most tempting · assemble three projects into a portfolio a hiring manager will actually open · tell the story of a project in two minutes and in ten · judge for yourself whether you are ready to apply.

Before you start: all of Part II. This chapter teaches almost nothing new. It uses Chapters 12 and 13 (SQL), 14 (cleaning), 17 and 18 (Python), 15 and 16 (visualization and Power BI), 20 (automation), 21 and 22 (statistics), 23 (metrics), 24 and 25 (requirements and stakeholders), and 26 (the repository this all lives in).

Time needed: 8–12 hours for the project, spread over a week. The reading is about an hour.

Tools: PostgreSQL or MySQL, Python with pandas, Power BI Desktop or any BI tool, Git. Nothing new.

Practice data: `riverstone_full`, the three-year database from Chapter 14 onward: 5,027 customer records, 116,194 orders, 209,006 order lines, 2023 to 2025. Every number and every output in this chapter was produced by running the query or the script shown, on PostgreSQL 16 and Python 3.11.

Why this matters

Seventeen chapters of Part II each taught one thing well. A hiring manager will not ask you about one thing. They will ask you to talk about something you built, and then they will find out, in about four minutes, whether you understand it.

This chapter is one worked pass through the whole analyst arc on a single question, and then a second half that no other chapter in this book covers: what happens to that work when somebody else looks at it.

The two halves belong together for a reason that becomes clear about halfway through. A portfolio is, by construction, a highlight reel. You choose what goes in it. That choice is the same choice you make when you decide which numbers go in a memo, and it is the point where analysts most often stop

being honest without ever deciding to be dishonest. Chapter 22 promised that this chapter would deal with selective reporting. It does, in section 27.8, using a real finding from this chapter's own project that did not survive being checked.

In plain English

A portfolio is not a gallery. It is evidence.

A gallery shows finished things, lit well, from their best angle. Evidence is different: it has to survive somebody picking it up and turning it over. When a carpenter applies for work, they do not bring a photograph of a finished house. They bring a joint. The joint is small, it can be examined from every side, and it shows whether the person can cut to a line.

Your portfolio projects are joints. Small enough to be examined, honest enough to survive it. The question a hiring manager is really asking is not "can you do impressive things?" It is "if I give you a number to produce, will it be right, and will you tell me when it isn't?"

27.1 A question worth putting in a portfolio

Most portfolio projects fail before any code is written, because the question was never worth answering.

Three tests. A question belongs in a portfolio if it passes all three:

Test	Why it matters	A question that fails
There is a decision behind it	Without one, no finding can be right or wrong, so nothing you conclude can be judged	"Exploratory analysis of the sales dataset"
The answer could come out either way	If you know the answer before you start, you are illustrating, not analyzing	"Show that revenue grew in 2025"
Somebody would be annoyed if you got it wrong	That is what makes the care visible	"Top 10 products by revenue"

The question this chapter answers came from a real conversation of the kind Chapter 24 taught you to have. Vikram Singh, the key accounts sales manager, said this at a Monday review:

"We give Wholesale much deeper discounts than anyone else and they buy far more. Should we be discounting harder in Retail to grow it the same way?"

That passes all three tests. There is a decision behind it, with rupees attached. The answer could be yes or no. And getting it wrong is expensive in both directions: discount when you should not and you give away margin, refuse when you should have and you leave growth on the table.

Chapter 25 would call this an unwritten requirement, so before touching the database, write down what has to be true for an answer to be usable:

This project	
The claim to test	Deeper discounts cause customers to order more
Unit of analysis	One customer, one year
Period	Calendar 2025, with 2024 held back to check whatever we find
Measures	Orders placed, and net revenue after discount
What would change the recommendation	Evidence that the same discount depth produces different order counts in customers who are otherwise alike
Out of scope	Whether discounting wins <i>new</i> customers. Different question, different data

That last row is the one people skip, and it is the row that stops a project sprawling.

The plan, which is also the shape of the rest of this chapter:

1. **SQL.** Get the headline number out of the database.
2. **Cleaning.** Deal with what the data does to that number, and record the decisions.
3. **The check.** Find out whether the headline survives being split.
4. **Python.** Make the whole thing re-runnable, so the decisions can be argued with.
5. **The dashboard.** One page that serves the decision, not the analysis.
6. **The memo.** What Vikram actually receives.

The analyst arc, and what each stage produced in this chapter's project

Nothing here is new. The capstone is the order, and the stage most people skip.



Stage 3 is the one that separates a capstone from a tutorial. It is also the one nobody can tell you skipped.

Figure 27.1 – The arc is not new. The order is the lesson, and stage 3 is the one that decides whether the project is honest.

27.2 SQL: the headline (a recap, not a re-teach)

Chapter 13 taught every piece of this query. Nothing here is new; the point is the order things happen in.

Start with the background question, because a claim about discounting should begin by checking whether discounting has moved at all:

```

SELECT EXTRACT(YEAR FROM o.order_date)::int AS year,
       ROUND(100 * (1 - SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100))
                / SUM(oi.quantity * oi.unit_price)), 2) AS discount_pct,
       ROUND(100 * (SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100))
                - SUM(oi.quantity * p.unit_cost))
                / SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)), 1) AS
margin_pct
FROM   orders AS o
JOIN   order_items AS oi ON oi.order_id = o.order_id
JOIN   products AS p ON p.product_id = oi.product_id
WHERE  o.status <> 'Cancelled'
GROUP BY year ORDER BY year;

```

year	discount_pct	margin_pct
2023	3.49	21.1
2024	3.43	24.7
2025	3.51	27.5

(3 rows)

Company-wide discounting has not moved in three years: 3.49%, 3.43%, 3.51%. Worth knowing before anybody claims discounting is "getting out of hand", and it takes one query.

Now the question itself. One row per customer for 2025, then the two groups compared:

```

WITH customer_year AS (
  SELECT o.customer_id,
         COUNT(DISTINCT o.order_id) AS orders,
         SUM(oi.quantity * oi.unit_price) AS list_revenue,
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS net_revenue
  FROM   orders AS o
  JOIN   order_items AS oi ON oi.order_id = o.order_id
  WHERE  o.status <> 'Cancelled'
         AND o.order_date >= DATE '2025-01-01' AND o.order_date < DATE '2026-01-01'
  GROUP BY o.customer_id
)
SELECT CASE WHEN 100 * (1 - net_revenue / list_revenue) >= 5
          THEN '5% or deeper' ELSE 'under 5%' END AS discount_band,
       COUNT(*) AS customers,
       ROUND(AVG(orders), 1) AS avg_orders,
       ROUND(AVG(net_revenue)) AS avg_revenue
FROM   customer_year
GROUP BY discount_band
ORDER BY discount_band;

```

discount_band	customers	avg_orders	avg_revenue
5% or deeper	681	14.8	459662
under 5%	3918	9.3	212765

(2 rows)

How it works, and why it is built this way:

- **The CTE computes the customer's own discount rate**, rather than averaging the discount percentages on their lines. Averaging percentages weights a ₹2,000 line the same as a ₹200,000 one. Dividing total net revenue by total list revenue weights by money, which is what the question means.

- `COUNT(DISTINCT o.order_id)` is necessary because the join to `order_items` puts a customer's order on the table once per line. This is Chapter 13's fan-out trap, and it is the single most common way this query goes wrong.
- The `WHERE` clause uses a half-open date range, `>= 2025-01-01` and `< 2026-01-01`, rather than `BETWEEN`. Chapter 13 section 13.4 explains why: `BETWEEN` on a timestamp column silently drops the last day.
- `status <> 'Cancelled'` is a business rule, not a technical one, and it is the first entry in the decision log below.
- `>= 5` is a line the analyst chose. Remember that. It becomes important in section 27.5.

The headline, in one sentence: customers on discounts of 5% or deeper placed 14.8 orders in 2025 and were worth ₹459,662 each, against 9.3 orders and ₹212,765 for everyone else. That is 59% more orders and more than double the revenue.

If this were a portfolio project written by most people, that sentence would be the finding, the chart would show those two bars, and the project would be finished. Hold on to it. It is wrong, and section 27.4 is where it comes apart.

27.3 Cleaning: the decisions, and whether they mattered

Chapter 14 taught the techniques. What a portfolio needs, and what Chapter 14 section 14.9 called the cleaning log, is the record of what you decided and why.

This dataset has four known problems. Each one is a decision, not a fix:

What is in the data	The decision	Why
4,766 cancelled orders	Excluded	A cancelled order is not revenue and was never a purchase decision
48 duplicate customer records, the same business entered twice with a name variant, holding 621 orders between them	Merged into their originals, then the analysis re-run both ways	Two records for one business understate that business's order count and inflate the customer count
100 customers with no city	Kept	The question does not use city. Dropping rows to tidy a column you do not need is how analyses lose data for no reason
3,414 orders with no sales rep, and 60 Pending orders dated before June 2025 that were never updated	Kept, flagged in the memo	Neither affects a customer-level revenue measure. Both are worth telling the source system's owner about

The duplicate decision is the interesting one, because it is the kind that sounds important. Chapter 14 section 14.5 built the matching. The honest thing to do with a decision like this is to measure it, which takes one extra run:

```
as loaded:
      customers  avg_orders  avg_revenue
deep           681        14.8    459662
shallow        3918         9.3    212765

duplicates merged into their originals:
      customers  avg_orders  avg_revenue
deep           674        15.0    464414
shallow        3881         9.3    214797
```

Merging the duplicates moved the deep-discount group from 14.8 orders to 15.0 and from ₹459,662 to ₹464,414. The conclusion is untouched.

That is worth writing down, and most people do not. "I cleaned the data" is an assertion. "I merged 48 duplicate customer records holding 621 orders, and it changed the headline by 0.2 orders and 1%" is evidence, and it takes one paragraph. It also tells a reviewer something more useful than the number itself: that you check whether your own work mattered.

The analysis below uses the merged version, because it is more correct, even though it makes no difference. Being right for the right reason is worth the extra hour when somebody may audit it.

27.4 The check that changes the answer

Here is the discipline Chapter 22 section 22.5 asked for: before believing that A causes B, look for the thing that could cause both.

The candidate here appears the moment you ask the question out loud. **Who gets 5% discounts at Riverstone?** Chapter 3 said it: the discount policy is tiered, and Wholesale buys in crate quantities. Split the same two bands by segment:

```
WITH customer_year AS (
  SELECT o.customer_id,
         COUNT(DISTINCT o.order_id) AS orders,
         SUM(oi.quantity * oi.unit_price) AS list_revenue,
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS net_revenue
  FROM   orders AS o
  JOIN   order_items AS oi ON oi.order_id = o.order_id
  WHERE  o.status <> 'Cancelled'
  AND    o.order_date >= DATE '2025-01-01' AND o.order_date < DATE '2026-01-01'
  GROUP BY o.customer_id
)
SELECT c.segment,
       CASE WHEN 100 * (1 - cy.net_revenue / cy.list_revenue) >= 5
            THEN '5% or deeper' ELSE 'under 5%' END AS discount_band,
       COUNT(*) AS customers,
       ROUND(AVG(cy.orders), 1) AS avg_orders,
       ROUND(AVG(cy.net_revenue)) AS avg_revenue
FROM   customer_year AS cy
JOIN   customers AS c ON c.customer_id = cy.customer_id
GROUP BY c.segment, discount_band
ORDER BY c.segment, discount_band;
```

The CTE is character for character the one from section 27.2. A CTE lives only for the statement that defines it, so it is repeated rather than reused. Once you are repeating it for the third time, it wants to be a view, which is what Chapter 13 section 13.2 did with `sales_lines`.

segment	discount_band	customers	avg_orders	avg_revenue
Hospitality	5% or deeper	9	1.2	16789
Hospitality	under 5%	1363	9.1	203467
Retail	5% or deeper	10	1.1	15020
Retail	under 5%	2555	9.3	217725
Wholesale	5% or deeper	662	15.2	472400

(5 rows)

Read the bottom of that table first. **Wholesale has no "under 5%" row at all.** Every Wholesale customer in 2025 sits at 5% or deeper, and 662 of the 681 deep-discount customers in the whole company are Wholesale.

So the two groups in section 27.2 were not "customers on deep discounts" and "customers on shallow discounts". They were "Wholesale" and "everyone else", wearing a different label. The headline said nothing about discounting. It said that Wholesale buys in bulk, which everybody already knew.

Look at the two remaining rows and it gets worse for the original claim. In Retail, the ten customers who did reach 5% averaged **1.1 orders** against 9.3 for everyone else. In Hospitality, 1.2 against 9.1. Those are customers who placed one big order and took a volume discount on it, not customers the discount made loyal.

The fair comparison is inside a single segment, where the customers are alike in the way that matters. Wholesale is the only segment with enough spread to try it:

```
WITH customer_year AS (
  SELECT o.customer_id,
         COUNT(DISTINCT o.order_id) AS orders,
         SUM(oi.quantity * oi.unit_price) AS list_revenue,
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS net_revenue
  FROM   orders AS o
  JOIN   order_items AS oi ON oi.order_id = o.order_id
  JOIN   customers AS c ON c.customer_id = o.customer_id
  WHERE  o.status <> 'Cancelled'
         AND o.order_date >= DATE '2025-01-01' AND o.order_date < DATE '2026-01-01'
         AND c.segment = 'Wholesale'
  GROUP BY o.customer_id
), banded AS (
  SELECT orders, net_revenue,
         100 * (1 - net_revenue / list_revenue) AS discount_pct,
         NTILE(4) OVER (ORDER BY 100 * (1 - net_revenue / list_revenue)) AS quartile
  FROM   customer_year
)
SELECT quartile,
       COUNT(*) AS customers,
       ROUND(AVG(discount_pct), 1) AS avg_discount_pct,
       ROUND(AVG(orders), 1) AS avg_orders,
       ROUND(AVG(net_revenue)) AS avg_revenue
FROM   banded
GROUP BY quartile
ORDER BY quartile;
```

quartile	customers	avg_discount_pct	avg_orders	avg_revenue
1	166	7.9	14.1	431573
2	166	8.5	16.7	519663
3	165	9.0	16.4	511059
4	165	9.7	13.6	427265

(4 rows)

How it works:

- **The CTE is the one from section 27.2** with two lines added: a join to `customers` and `AND c.segment = 'Wholesale'`. `customer_year` now holds only the 662 Wholesale customers, and nothing else in the query changes.
- **`NTILE(4) OVER (ORDER BY ...)`** is Chapter 13 section 13.7's window function. It sorts those customers by their own discount rate and cuts them into four groups of equal size, numbering them 1 to 4. It is used here rather than fixed discount bands because the whole range is between 6.7% and 12%, so fixed bands would put almost everyone in one of them.
- **The same expression appears in the `SELECT` list and in the `ORDER BY` inside `OVER`**. SQL will not let a window function refer to an alias defined in the same `SELECT`, which is why it is written twice. A second CTE would avoid the repetition and is worth it once the expression is longer than this.
- **`banded` exists as a separate CTE** because you cannot group by a window function's result in the same query that computes it. Windows are evaluated after `GROUP BY`, so the quartile has to be finished before the outer query can aggregate on it.
- **`AVG(discount_pct)` in the output** is there to prove the quartiles are what you think they are. Printing the thing you sorted by is cheap and catches a whole class of silent error.

The shallowest quartile, averaging 7.9% discount, placed **14.1** orders. The deepest, at 9.7%, placed **13.6**. The middle two are higher than both ends. There is no ladder here. Nearly two extra percentage points of discount, worth real money at Wholesale volumes, bought nothing.

Chapter 21 and Chapter 22 supply the last step, because "14.1 against 13.6" is a difference and the question is whether it is a real one:

```
Q4 - Q1 = -0.48 orders; 95% CI [-1.89, 0.92]; p = 0.50
within Wholesale: pearson r = -0.081 (p = 0.038), n = 662
```

Two things to read carefully, and the second is the one that separates an analyst from someone who has learned a formula.

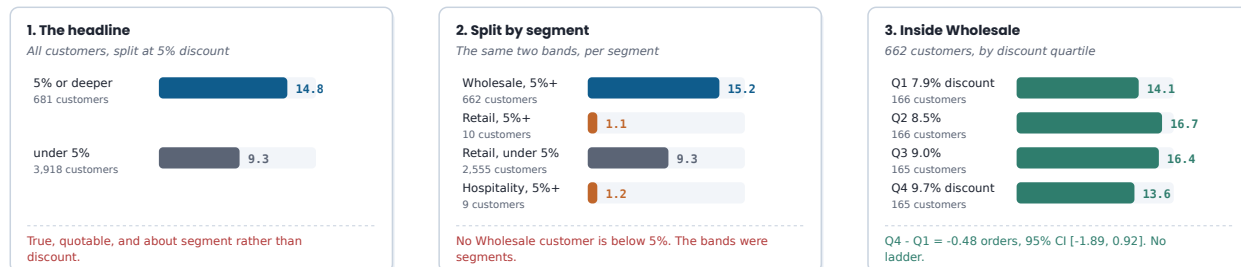
The confidence interval straddles zero, from 1.89 fewer orders to 0.92 more. Chapter 22 section 22.2 is precise about what that means: the data is consistent with a small effect in either direction and with no effect at all, so it does not support a claim in either direction.

The correlation is statistically significant and practically nothing. An r of -0.081 with $p = 0.038$ clears the conventional 5% bar, and it points the opposite way from the claim. With 662 customers, a tiny wobble becomes "significant". Chapter 22 section 22.3 warned about exactly this: significance is a statement about sample size as much as about effect. Reporting "significant negative relationship

between discount depth and order frequency" would be technically true and would badly mislead the reader. The honest sentence is: *within Wholesale, discount depth explains essentially none of the variation in how often a customer orders.*

The same data, asked three ways. Only the third one answers the question.

Average orders per customer, 2025. Every bar is true.



Deeper discounts do not buy more orders. Panel 1 is what a portfolio shows when the analyst stops one query early.

Figure 27.2 – Every bar in all three panels is true. Only the third panel answers the question that was asked.

The answer to Vikram's question is no, and the reason is more useful to him than the answer: discount depth at Riverstone is a label for what segment a customer is in, not a lever that changes how they behave.

27.5 Python: making the analysis arguable

Everything above is six queries in a file. That is fine for finding the answer and useless for the conversation that follows, which always starts with "what if you had drawn the line somewhere else?"

Chapters 17 and 18 taught the pieces. Here they are as one script, which is the artifact that goes in the repository. It has two functions, and they are shown one at a time because they do two different jobs.

The first builds the table. One row per customer, which is the grain the question needs:

```

"""Does a deeper discount buy Riverstone more orders? One year, one answer."""
import pandas as pd

DATA = "."

def customer_year(year=2025, segment=None, min_orders=1):
    """One row per customer: orders placed, list revenue, net revenue, discount rate."""
    orders = pd.read_csv(f"{DATA}/orders.csv", parse_dates=["order_date"])
    items = pd.read_csv(f"{DATA}/order_items.csv")
    customers = pd.read_csv(f"{DATA}/customers.csv")

    orders = orders[(orders.status != "Cancelled") & (orders.order_date.dt.year == year)]
    lines = items.merge(orders[["order_id", "customer_id"]], on="order_id")
    lines["list_revenue"] = lines.quantity * lines.unit_price
    lines["net_revenue"] = lines.list_revenue * (1 - lines.discount_pct / 100)

    per_customer = (lines.groupby("customer_id")
                    .agg(orders=("order_id", "nunique"),
                        list_revenue=("list_revenue", "sum"),
                        net_revenue=("net_revenue", "sum"))
                    .reset_index()
                    .merge(customers[["customer_id", "segment"]], on="customer_id"))
    per_customer["discount_pct"] = 100 * (1 - per_customer.net_revenue / per_customer.list_revenue)

    if segment is not None:
        per_customer = per_customer[per_customer.segment == segment]
    return per_customer[per_customer.orders >= min_orders]

```

How it works, line by line:

- **DATA = "."** is the folder holding the CSV files, named once at the top so there is one place to change it. A path repeated in four `read_csv` calls is four places to get it wrong.
- **parse_dates=["order_date"]** makes pandas read that column as a date rather than text. Without it, `.dt.year` fails and any date comparison compares strings, which Chapter 18 section 18.3 shows going wrong quietly.
- **orders[(orders.status != "Cancelled") & (orders.order_date.dt.year == year)]** applies the two filters that the SQL `WHERE` clause applied. The parentheses around each condition are required: `&` binds tighter than `!=` in Python, so leaving them out is a `TypeError` rather than a wrong answer, which is the good kind of mistake.
- **items.merge(orders[["order_id", "customer_id"]], on="order_id")** joins lines to orders. The `on` argument names the column both frames are matched on, here the order key, and because a merge is an inner join by default, lines whose order was filtered out disappear. That is the cancelled-order rule and the year filter arriving in one step. Selecting only those two columns keeps the result narrow; `merge` would otherwise carry every order column along.
- **lines["list_revenue"]** and **lines["net_revenue"]** are the same two expressions as the SQL, computed per line so they can be summed per customer.
- **.groupby("customer_id").agg(orders=("order_id", "nunique"), ...)** is the pandas form of `GROUP BY` with named outputs: each named argument is `new_column=(source_column, function)`. `"nunique"` is `COUNT(DISTINCT)`. Using `"count"` here would count lines and overstate every customer's order count, which is the fan-out trap of section 27.2 wearing different clothes.
- **.reset_index()** turns the grouping key back into an ordinary column, so the `merge` that follows can join on it.

- `100 * (1 - net_revenue / list_revenue)` is computed after the sums, not averaged across lines, for the money-weighting reason in section 27.2.
- **The last three lines apply `segment` and `min_orders`**. They are last on purpose: filtering customers out after the totals are built means the totals are never affected by the filter. Filtering the lines first would have changed each customer's discount rate.

The second function reports. It takes the table and answers the question:

```
def discount_report(year=2025, segment=None, band_cut=5.0, min_orders=1):
    """Average orders and revenue for customers above and below one discount line."""
    people = customer_year(year=year, segment=segment, min_orders=min_orders)
    people = people.assign(
        band=people.discount_pct.ge(band_cut).map({True: f"{band_cut:g}% or deeper",
                                                    False: f"under {band_cut:g}%"})
    )
    return (people.groupby("band")
            .agg(customers=("customer_id", "size"),
                 avg_orders=("orders", "mean"),
                 avg_revenue=("net_revenue", "mean"))
            .round({"avg_orders": 1, "avg_revenue": 0}))

if __name__ == "__main__":
    print(discount_report())
```

	customers	avg_orders	avg_revenue
band			
5% or deeper	681	14.8	459662.0
under 5%	3918	9.3	212765.0

How it works, line by line:

- It calls `customer_year` rather than rebuilding the table, which is the whole reason for two functions: a chart, a statistical test, or a different cut can all start from the same table without any of them rewriting it.
- `.ge(band_cut)` is "greater than or equal to" as a method, giving a column of True and False. It is written this way rather than with `>=` so it can be chained straight into `.map`.
- `.map({True: ..., False: ...})` turns that into two readable labels, so the output says "5% or deeper" rather than "True". `{band_cut:g}` formats 5.0 as 5 rather than 5.0, so the label reads properly whatever the setting is.
- `.assign(band=...)` returns a new frame with the column added rather than modifying the existing one, which keeps the function safe to call twice in a row, as the what-if below does.
- `.agg(customers=("customer_id", "size"), ...)` counts customers with "size" rather than "count", because "count" skips missing values and "size" does not. Here they agree; when a column has nulls they do not, and the difference is the kind that goes unnoticed for months.
- `.round({"avg_orders": 1, "avg_revenue": 0})` rounds each column to its own number of decimals. Rounding for display belongs at the end, never in the middle, so nothing downstream inherits a rounded number.
- `if __name__ == "__main__":` means the file can be imported by a notebook without running the report, and run from the command line when you want it. Chapter 17 section 17.8 introduced this.

The settings, and what happens if you change them

Setting	Default	What it does	What happens if you change it
year	2025	Which calendar year is analyzed	2024 re-runs the whole thing on a year you have not looked at. This is how you find out whether a finding is a fact or a fluke, and it costs one keystroke
segment	None	Restricts to one segment	"Wholesale" gives the fair comparison of section 27.4. None compares groups that differ in more than discount, which is what produced the wrong headline
band_cut	5.0	The discount rate that separates "deep" from "shallow"	Moving it changes the size of the gap you report. See the measurement below. This is the most dangerous number in the script, because it looks like a fact and is a choice
min_orders	1	Drops customers below this many orders	Raising it to 5 removes one-order customers, which raises both averages. Defensible if the question is about ongoing relationships, and it must be stated when it is used

A measured what-if: the line you drew

`band_cut` was set to 5 because Riverstone's approval policy has a step there, which is a real reason. Here is what happens if it moves to 3, which is also defensible:

```
print(discount_report(band_cut=5.0))
print(discount_report(band_cut=3.0))
```

	customers	avg_orders	avg_revenue
band			
5% or deeper	681	14.8	459662.0
under 5%	3918	9.3	212765.0

	customers	avg_orders	avg_revenue
band			
3% or deeper	840	12.9	392353.0
under 3%	3759	9.4	217362.0

At 5% the gap is **5.5 orders and ₹246,897**. At 3% the same data gives **3.5 orders and ₹174,991**. The headline shrank by more than a third because of a decision the analyst made and could have made either way, on data that did not change at all.

Nobody moved a number. Nobody was dishonest. This is Chapter 22 section 22.4's garden of forking paths, in one line of one function, and the reason it matters here is that a portfolio only ever shows one branch of that garden.

Two more, for the same reason:

```
print(discount_report(segment="Wholesale", band_cut=9.0))
print(discount_report(year=2024))
```

	customers	avg_orders	avg_revenue
band			
9% or deeper	244	14.5	451061.0
under 9%	418	15.6	484856.0
	customers	avg_orders	avg_revenue
band			
5% or deeper	602	13.4	396882.0
under 5%	3502	8.6	189632.0

The first confirms section 27.4 from a different angle: inside Wholesale, the more heavily discounted half orders **less** often, 14.5 against 15.6. The second re-runs the original headline on 2024 and gets the same shape, 13.4 against 8.6. So the headline is stable and still means nothing about discounting: 2024's Wholesale customers were also on deep discounts and also bought in bulk.

Stability and truth are different properties. A result that reproduces every year can still be answering a question you did not ask. That sentence is most of what section 27.8 is about.

27.6 The dashboard: one page, one decision

Chapter 15 gave the principles and Chapter 16 built the thing. The capstone question is narrower than either: **what should be on the page, given that a specific person has a specific decision to make?**

For Vikram's question, that is one page with four objects and nothing else:

Object	What it shows	Why it earns its place
One sentence at the top	"Discount depth tracks segment, not buying behavior. Within Wholesale, deeper discounts do not increase order frequency."	Chapter 15 section 15.6: the title is the finding, not the subject. A reader who stops here has the answer
Small multiples: orders per customer by discount quartile, one panel per segment	The three segments side by side, on a shared axis	This is the whole argument in one object. The Wholesale panel is flat; Retail and Hospitality have no deep-discount customers to compare
A slicer for year	2023, 2024, 2025	So the first question anyone asks, "is this just last year?", is answered by the reader rather than by an email to you
Margin by segment	Wholesale 18.6%, Hospitality 31.8%, Retail 30.2%	The cost side of the decision. Discounting Retail toward Wholesale depth is a margin decision before it is a growth one

And what is deliberately **not** on it: the company discount trend from section 27.2 (background, belongs in the memo), the confidence interval (a reader who needs it needs the memo), the customer-level table (a dashboard that invites browsing invites a different question), and any chart of revenue over time, which every dashboard acquires by gravity and which serves no decision here.

The test for a dashboard page, which is worth more than any styling rule: name the decision, name the person, and name what they would do differently for each thing the page could show. If a chart has no answer to the third one, take it off the page.

One detail from Chapter 16 that matters for a portfolio specifically: save the file in the `.pbip` project format, not `.pbix`. Chapter 26 section 26.9 explains why, and a reviewer who can read the model definition as text learns more about you in thirty seconds than the rendered page tells them in five minutes.

27.7 The memo

This is what Vikram receives. It is one page, it recommends **not** doing something, and the last section is the one that makes it trustworthy.

To: Vikram Singh, Sales Manager (Key Accounts); Anita Rao, Sales Head **From:** Analytics **Date:** 12 February 2026 **Re:** Should we discount harder in Retail?

Recommendation: no, not on the evidence that Wholesale grows faster. Discount depth at Riverstone is a marker of which segment a customer is in, not a lever that changes how often they order. Increasing Retail discounts would reduce margin with no support in three years of our own data for the growth it is meant to buy.

What the data shows. Customers on discounts of 5% or deeper placed 14.8 orders in 2025 against 9.3 for everyone else, which is where this question came from. But 662 of those 681 customers are Wholesale, and no Wholesale customer is below 5%, so that comparison is Wholesale against everyone else with a discount label on it. Inside Wholesale, where customers are comparable, the shallowest quarter of discounts (averaging 7.9%) placed 14.1 orders and the deepest (9.7%) placed 13.6. The difference is not distinguishable from zero.

The cost side. Wholesale runs at 18.6% gross margin against 30.2% for Retail. Moving Retail discounts up toward Wholesale depth is a decision to spend margin, and the growth case for it is not in this data.

What I checked that did not support the recommendation. The headline gap reproduces in 2024 (13.4 orders against 8.6), so it is stable, but stability does not make it causal: the same segment mix produces it. I also tested whether the effect appears within Retail and Hospitality; both have fewer than a dozen customers above 5%, and those few placed about one order each, so there is no usable comparison there rather than evidence of no effect. If Retail discounting were to be tried, this is the gap that would need filling.

What would change this recommendation. A deliberate test: offer a deeper discount to a randomly chosen group of Retail customers for two quarters and compare order frequency against a held-back group. That is the only way to answer a causal question with this data, and it would cost far less than a policy change. Chapter 22's warning applies: a difference found by slicing is a hypothesis, not a finding.

Two data problems worth fixing regardless. 48 customer records are duplicates of existing businesses, and 3,414 of 2025's orders have no sales rep recorded. Neither changes this analysis. Both distort any per-rep or per-customer report built on the CRM.

Four things about that memo are worth copying, and none of them are about writing style.

1. **The recommendation is in the first line**, with the reason immediately after. Chapter 24 section 24.4: a reader who stops after two sentences should still act correctly.
2. **The headline that turned out to be misleading is stated, not hidden**. It is why the question exists, and a reader who has seen that number elsewhere needs to know you saw it too.
3. **There is a section for what did not support the conclusion**. This is the part that almost nobody writes, and it is the part that makes the rest believable.
4. **"What would change this recommendation" is a real proposal**, costed relative to the alternative. A memo that only says no gets ignored; a memo that says no and names the cheap way to find out gets acted on.

27.8 Selective reporting, and why a portfolio is where it starts

Chapter 22 promised this section. It is the most important one in the chapter.

Selective reporting is not lying. Every number you publish will be true. Selective reporting is choosing which true numbers the reader sees, in a way that leads them somewhere the full set would not. It almost never feels like a decision. It feels like editing.

This chapter's own project is a worked example. Section 27.2 produced a clean, quotable, entirely true sentence: *customers on 5% or deeper discounts placed 59% more orders and were worth more than twice as much*. A chart of those two bars would look professional. A portfolio built around it would read well. And a reader acting on it would push discounts into Retail and lose margin for nothing.

Nothing in that project would have been fabricated. The analyst would have stopped one query early.

The five moves, and what each looks like in a portfolio

The move	What it looks like	What it looks like in a portfolio
Stopping at the flattering result	You do not run the split that would complicate it	The project has one finding and no section about what you checked
Choosing the cut that works	Moving <code>band_cut</code> from 3 to 5 because the gap is bigger	A threshold with no stated reason, which is why section 27.5 measured the effect of moving it
Reporting the slice that survived	Twelve segments were tested; the one with a result is the project	"Analysis of premium Hospitality customers in the western region" with no mention of the eleven other slices
Choosing the window	The trend holds from 2023, so the chart starts in 2023	Any time series whose start date is not the start of the data, unexplained
Dropping the inconvenient rows	Outliers removed once, after seeing the result	A cleaning step that appears in the code but not in the write-up

Chapter 22 section 22.4 called the underlying mechanism the garden of forking paths: a series of small, reasonable decisions that each nudge the answer. A portfolio is that garden with the wrong turnings mown over. The reader sees one path and cannot tell how many there were.

Why it is worse in a portfolio than at work

At work there is friction. Somebody knows the domain and says "that can't be right for Retail". The number gets checked against a system. You are there in six months when the policy built on your analysis fails.

A portfolio has none of that. Nobody will ever check it. You will never see the consequence. The only thing standing between a portfolio and a highlight reel is a habit you decided to keep when nobody could tell.

That is also precisely why an interviewer values the evidence of the habit so highly. Anyone can produce a finding. Very few candidates volunteer the check that nearly killed it.

The four things that fix it, and they are cheap

1. **Write the analysis plan before you run it.** Two paragraphs: the claim, the comparison that would test it, and what result would make you drop it. Commit it first, so its timestamp is before the results. Chapter 26 gives you a history that proves the order.
2. **Keep a "what I tried" section.** Every cut you ran, including the ones that showed nothing. In this project that section holds the 2024 replication, the within-Wholesale quartiles, the `band_cut` sensitivity, and the two segments with too few customers to say anything. That is four honest lines.
3. **Report the denominator of your search.** "I looked at six segment cuts and one was interesting" is a completely different claim from "the Wholesale cut is interesting", and Chapter 22 section 22.4 explains why the first one needs a correction and the second one hides that it needs one.
4. **Name the null result in the summary, not the appendix.** In this chapter's memo it is a heading. A null result buried in a footnote has been reported and concealed at the same time.

The version of this that shows up in interviews

The question is usually "tell me about a project you're proud of", and the trap is that pride pushes you toward the highlight reel.

A weak answer describes what was built. A strong answer describes what was nearly believed:

"The headline was that deep-discount customers order 59% more. I almost wrote that up, and then I split it by segment and found that 662 of the 681 deep-discount customers were Wholesale, and no Wholesale customer was below 5%. So the comparison was really Wholesale against everyone else. Inside Wholesale there's no relationship at all, and the confidence interval straddles zero. The recommendation flipped from 'discount harder in Retail' to 'don't, and here's the test that would actually answer it.'"

That is forty-five seconds. It demonstrates SQL, segmentation, confounding, confidence intervals, and business judgment, and none of them are claimed. It works because it tells the story of a mind changing, and an interviewer's actual question, underneath the one they asked, is whether yours does.

One caution, because this advice gets overapplied. Reporting what did not work is not the same as having nothing that did. A project whose entire story is caveats reads as someone who could not get to an answer. The shape that works is a clear recommendation, reached the hard way, with the hard part visible.

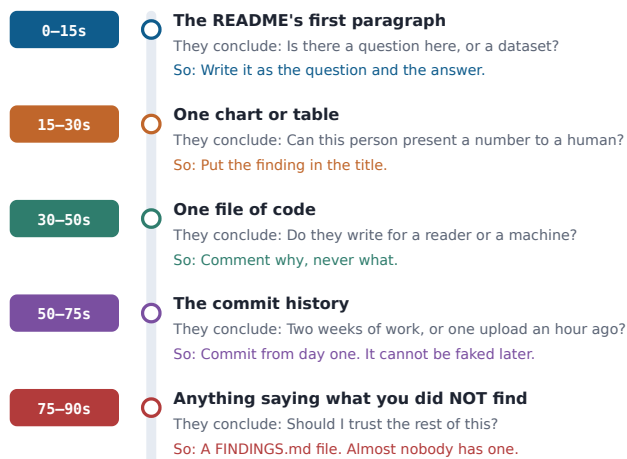
27.9 What a hiring manager does with your repository

They open it, they spend ninety seconds, and then they either read properly or they close it. Knowing the order those ninety seconds happen in is worth more than another week of work on the analysis.

Seconds	What they look at	What they conclude
0–15	The README's first paragraph	Whether there is a question here, or a dataset
15–30	One chart or table, whichever appears first	Whether you can present a number to a person
30–50	One file of code, usually the one with the most interesting name	Whether you write for a reader or for a machine
50–75	The commit history	Whether this was built over two weeks or uploaded in one commit an hour ago
75–90	Whether anything says what you did not find	Whether to trust the rest

Ninety seconds: what a hiring manager reads, in order

Then they either read properly or they close the tab. The order is predictable, so build for it.



Three of the five are writing, and one is a habit you cannot add at the end. Only one is the analysis.

Figure 27.3 — Three of the five moments are writing, and one is a habit that cannot be added afterward.

Three things follow from that table, and all three are cheap.

The first paragraph of the README is the most valuable writing in your portfolio. It gets read every time, by everyone, and it is usually a description of the dataset. It should be the question and the answer: "Riverstone's sales manager asked whether deeper discounts would grow Retail the way they appear to have grown Wholesale. They would not, and this repository shows why the appearance is a segment effect."

Commit history is a character reference you cannot fake after the fact. Chapter 26's habit of committing work as you finish it produces, without any extra effort, a record that says you worked steadily and thought in steps. One commit called "initial commit" containing an entire project says something else, and nothing in the README can correct it.

A **FINDINGS.md** beside the README, containing what you checked and what it showed, is the single highest-return file in a portfolio, because almost nobody has one and it answers the question that decides whether the reviewer trusts the rest.

27.10 Telling the story, in two minutes and in ten

Have both versions ready, because you will be asked for the short one and then interrupted into the long one.

The two-minute version has four parts and no code:

1. **The question and who asked it.** One sentence, naming a role. "The key accounts manager wanted to know whether deeper discounts would grow Retail."
2. **What you did.** Three steps at most, named not described. "Built a customer-year table from the order lines, compared discount bands, then split by segment."
3. **What you found, including the turn.** The version in section 27.8.
4. **What happened, or would happen, next.** "The recommendation was no, with a proposed two-quarter test on a random group of Retail customers."

The ten-minute version is the same four parts with the interviewer driving. Have these five ready, because they are what gets asked:

- **"How do you know the number is right?"** Name the reconciliation. Here: company net revenue matches the finance figure for 2025 to the rupee, and `COUNT(DISTINCT order_id)` rather than `COUNT(*)` because the join to lines multiplies rows.
- **"What would you do differently?"** Have a real answer. Here: write the analysis plan before running the first query, because the segment split should have been in the plan and instead it was a save.
- **"What was hardest?"** Not a modesty question. It is asking what you found difficult, which tells them your level. "Knowing that a significant correlation of -0.08 was not worth reporting as a finding" is a better answer than any tooling difficulty.
- **"Walk me through this bit of code."** They will pick something. You must be able to explain every line of every file in your portfolio, which is the real reason Chapter 26 section 26.11 says to read what an assistant gives you before you keep it.
- **"What if I told you the result is wrong?"** They are testing whether you fold or get defensive. The right response is neither: ask which part, and say what evidence would change your mind. You already wrote that section in the memo.

On presenting the dashboard, one rule: do not narrate the page. Say the decision it serves, then be quiet and let them look. Analysts talk over their own charts more than any other mistake in this room.

27.11 What job-ready actually looks like

Not "have I finished Part II". The honest test is whether you could survive the first month, and it has six parts:

Ready means	Concretely
You can get data out	Given a schema you have not seen, you can write a multi-table query with the right grain and know when a join has fanned out
You can tell whether a number is right	You reconcile against something already trusted before sending anything, and you can say what the number excludes
You can turn a request into a question	You ask what decision it serves and what the words mean before opening a tool
You can make it repeatable	Your analysis is a script with parameters, in a repository, that someone else can run
You can present to a non-technical reader	One page, finding first, limitations included, no jargon that does not pay its way
You know what you do not know	You can name three things you would need help with, which is the answer to "any questions for us?"

Three projects is the right number. **One deep**, like this chapter's, with the full arc and a FINDINGS file. **One narrow and technical**, a hard SQL problem or a cleaning job on real, messy data, which shows craft. **One visual**, a dashboard built for a named decision. More than three dilutes; fewer than three is a sample of one.

What does not make you job-ready, despite feeling like it does: finishing courses, adding tools to a CV, a project whose data came pre-cleaned from a competition site, and a dashboard with no decision behind it. Every one of those is a thing you did. None of them is evidence about what you would do.

Common mistakes and how to spot them

Mistake	Symptom	Fix
A project with no decision behind it	the README describes a dataset rather than a question	write the question and the person who asked it before writing any code
Stopping at the first clean result	one finding, no section on what you checked	run the split that would complicate it, then report what happened either way
Comparing groups that differ in more than one way	a large, tidy gap between two groups	ask what else is true of the group, and compare inside it
Reporting significance without effect size	"significant relationship" on thousands of rows	give the size and the interval. An r of -0.08 is significant at $n = 662$ and means nothing
A threshold with no stated reason	a <code>band_cut</code> , a top-N, a date range, appearing without explanation	give the reason, and measure what happens if it moves

Mistake	Symptom	Fix
Averaging percentages	discount rates or margins that do not tie back to the totals	divide the summed numerator by the summed denominator
<code>COUNT(*)</code> after joining to a detail table	order counts far higher than the business would recognize	<code>COUNT(DISTINCT order_id)</code> , and reconcile against a known total
Cleaning decisions made but not recorded	a reviewer cannot tell what was dropped or why	a decision log, with a measurement of whether it changed the answer
A dashboard that shows everything	four charts nobody has ever acted on	name the decision and the person; remove anything that does not change what they do
A memo that only reports the upside	it reads well and nobody believes the next one	a "what did not support this" section, in the body, not an appendix
Presenting the project as a tour of the code	the interviewer's attention goes before the finding does	question, what you did, what you found, what next. Four parts, two minutes
One commit called "initial commit"	the history says the work happened in an hour	commit as you go, from the first day. It cannot be reconstructed later
Ten shallow projects	none of them can survive a follow-up question	three: one deep, one technical, one visual
Code in the portfolio you cannot explain	a stall on "walk me through this bit"	read every line of anything you keep, whoever or whatever wrote it
Claiming a tool you used once	the follow-up question arrives immediately	list what you would be comfortable being tested on. Nothing else

In the real world: the finding Farah nearly led with

Farah Khan's first portfolio piece was the one Chapter 9 followed her building: hospitality orders before the wedding season, this year against last.

The finding was clean. Hospitality customers who ordered in the eight weeks before the season placed more orders across the rest of the year than those who did not. She had the query, a chart, and a sentence: *early-season buyers are worth more, so the team should push hospitality outreach into August.*

Anita Rao asked one question about it: "worth more than whom?"

The comparison group was hospitality customers who had not ordered in that window. Some of them were new that year and had not existed in August. Some were caterers who had closed. The group Farah had labeled "did not order early" included every account that was not really an account at all, and their low order counts were doing most of the work in the gap.

When she restricted both groups to customers who had ordered at least once in each of the two previous years, the gap shrank to something she described, in the version she kept, as "smaller than I can distinguish from noise with 140 customers".

The project she put in her portfolio was the second version, and it kept the first one. The README's second paragraph said what the original comparison had been, why it was wrong, and what the corrected version showed. She added a `FINDINGS.md` with four lines in it, one of which recorded a cut that showed nothing at all.

The part worth taking from this is not that she caught the error. It is what the correction did to the project. The original was a finding anyone could have produced, and a reviewer would have had no way to judge it. The corrected version was evidence of a specific, checkable skill: she knew that a comparison group is a choice, and she wrote down what happened when she chose differently.

Her practice log for that week has one line in it: *"the good version of the project was the one that said less."*

Tools

Tool	What it is for in a portfolio
Git and GitHub	Where the portfolio lives. The commit history is part of the evidence, which is why Chapter 26 comes before this chapter
A README and a FINDINGS.md	The two files that decide whether anything else gets read
PostgreSQL or MySQL	The queries. Keep the <code>.sql</code> files; a reviewer reads them more often than the Python
Python with pandas	Turning the analysis into something re-runnable with parameters
Jupyter	Fine for exploring. Clear outputs before committing (Chapter 26), and do not make a notebook the deliverable: notebooks reward narration and hide structure
Power BI Desktop, Tableau Public, or Looker Studio	The visual project. Tableau Public and Looker Studio produce a link, which is worth a great deal when a reviewer will not install anything
Power BI .pbip format	So a reviewer can read the model as text (Chapter 16, Chapter 26)
GitHub Pages	A one-page site over the repository, if you want a portfolio index. Optional, and never a substitute for the README
A plain text file of the questions you were asked	After every interview. It is the highest-value study document you will ever own, and nobody keeps one

The project: build the portfolio

This is the capstone of Part II. Budget a week, not an evening.

1. Choose three questions, not three datasets. One for each project: the deep one, the technical one, the visual one. Each must pass the three tests in section 27.1. Write each as a sentence a named role would say out loud. If you cannot name the role, the question is not ready.

2. Write the analysis plan for the deep one, and commit it first. Two paragraphs: the claim, the comparison that would test it, and what would make you drop it. Committing it first is what makes the claim credible later.

3. Run the full arc on the deep project. SQL to a customer-level or order-level table; cleaning with a decision log, including one decision you measure the effect of; the check that could overturn your headline, run whether or not you want to; a Python script with parameters and a settings table in the README; a one-page dashboard for a named decision; a one-page memo with a "what did not support this" section.

4. Write the FINDINGS.md. Every cut you ran, one line each, including the ones that showed nothing. If this file has only successes in it, you have not finished step 3.

5. Make each repository pass the ninety-second test. Give all three to someone who has not seen them, say nothing, and watch. Where they hesitate is where the README is wrong. Chapter 26 section 26.9's fresh-clone test applies to all three.

6. Record the two-minute version. Out loud, on your phone, for each project. Listen to it once. Almost everyone discovers they spent ninety seconds on what they built and ten on what they found, and that the fix is to start from the last sentence.

Stretch. Swap portfolios with someone else at your level and review theirs the way section 27.9 describes: ninety seconds, then write down what you concluded at each stage. Being on the other side of that table once is worth more than a month of polishing.

What "done" looks like: three URLs, each of which answers a question a named person would care about, where a stranger can see what you found, what you checked, and what you did not find, without asking you anything.

You've got it when...

- You can look at a question and say whether it belongs in a portfolio, using the three tests, before writing any code.
- You write the analysis plan first, and you can prove you did.
- Your headline finding survives being split by the confounder a reader would name first, or you report what happened when it did not.
- You can tell the difference between a statistically significant result and an important one, and you say so in plain language.
- Every threshold in your work has a reason, and you have measured what happens when it moves.
- Your cleaning decisions are written down with the effect each had on the answer.
- Your memo has a section for what did not support your conclusion, and it is in the body.
- Your dashboard serves one named decision, and you can defend the removal of everything that is not on it.
- Your README's first paragraph is the question and the answer, and your repository has a FINDINGS file.

- You can tell a project in two minutes, ending on the turn rather than the technique.
 - You can say what you would do differently, what was hardest, and what you would need help with, without rehearsing.
-

Recap

A portfolio question needs three things: a decision behind it, an answer that could go either way, and a cost to getting it wrong. Everything else is a dataset with a chart on it.

The analyst arc runs SQL, cleaning, the check, Python, the dashboard, the memo. The SQL gets the headline; the cleaning log records the decisions and, where it is cheap, measures whether they mattered; the script makes the analysis arguable by turning the analyst's choices into parameters; the dashboard serves one decision for one person; the memo leads with the recommendation.

The check is the part that separates this from a tutorial. This chapter's headline, that deep-discount customers place 59% more orders, was true and meaningless: 662 of the 681 deep-discount customers were Wholesale and no Wholesale customer was below 5%, so the comparison was one segment against the rest. Inside Wholesale the relationship disappears, the confidence interval straddles zero, and a correlation of -0.08 reaches significance only because there are 662 customers. Discount depth was a label for segment, not a lever.

Selective reporting is what happens when that check is skipped, or run and quietly dropped. It is not lying; every published number stays true. The five moves are stopping at the flattering result, choosing the cut that works, reporting the slice that survived, choosing the window, and dropping the inconvenient rows. A portfolio is where they are most tempting, because nobody will ever check it and you will never see the consequence. Four cheap habits fix it: plan first and commit it, keep a "what I tried" file, report the denominator of your search, and put the null result in the summary rather than the appendix.

A hiring manager gives a repository about ninety seconds, in a predictable order: the README's first paragraph, one chart, one file of code, the commit history, and whether anything says what you did not find. Write the first paragraph as the question and the answer, commit as you work, and keep a FINDINGS file.

Tell a project in four parts: the question and who asked it, what you did, what you found including the turn, and what happened next. Have the two-minute version and the ten-minute version, and expect to be asked how you know the number is right, what you would do differently, what was hardest, to walk through some code, and what you would say if the result were wrong.

Job-ready is not a syllabus. It is six things: you can get data out, you can tell whether a number is right, you can turn a request into a question, you can make the work repeatable, you can present it to someone non-technical, and you can name what you do not know. Three projects, one of each kind, is the right size.

Practice exercises

Warm-up

1. For each, say which of section 27.1's three tests it fails: (a) "Exploring the Riverstone customer table"; (b) "Proving that our new packaging increased sales"; (c) "Monthly revenue by category, 2023 to 2025".
2. Section 27.2's query uses `COUNT(DISTINCT o.order_id)`. What number would `COUNT(*)` have produced instead, and would it have been higher or lower?
3. Why does the chapter compute a customer's discount rate as total net revenue over total list revenue, rather than averaging `discount_pct` across their order lines?
4. The memo has a section headed "What I checked that did not support the recommendation". Name two things that belong in it for this project.

Core

5. **Predict before running.** Section 27.5 showed `band_cut=5.0` and `band_cut=3.0`. Before running it, write down whether `band_cut=8.0` will make the gap in average orders larger or smaller than at 5.0, and why. Then run it and see.
6. Write the analysis plan for Vikram's question, as it should have been written before any query: the claim, the comparison that would test it, and the result that would make you drop it. Three sentences each at most.
7. A colleague shows you this: "Customers who open our marketing emails place 2.3 times more orders. We should send more emails." Name the confounder, and describe the comparison you would run instead.
8. Section 27.4 reports `r = -0.081` with `p = 0.038`. Write the one sentence you would put in a memo about that number, and the one sentence you would put in it if the sample had been 40 customers instead of 662 with the same `r`.
9. Take one project you have already built in Part II. Write its README first paragraph in the section 27.9 form: the question, who asked it, and the answer, in under sixty words.
10. Write the four lines of a `FINDINGS.md` for this chapter's project.

Stretch

11. Design the two-quarter Retail discount test the memo proposes. Say who is in each group, what you would measure, how long you would run it, and what result would justify the policy change. Chapter 22 section 22.6 is the relevant material.
12. The Wholesale quartile table shows Q2 and Q3 above both Q1 and Q4. Give two explanations for that shape, and say what you would look at to tell them apart.
13. Rewrite this chapter's memo for a reader who is not Vikram but the finance manager, whose decision is whether to change the approval thresholds. What changes, and what stays?

Think about it (no calculation needed)

- l4. Section 27.8 argues that a portfolio has no friction: nobody checks it and you never see the consequence. If that is true, why would an interviewer believe your FINDINGS file, which you also wrote with nobody checking?
 - l5. This chapter's project ends in a recommendation not to do something, supported by a null result. Career advice usually says to show impact. How do you present a project whose value was preventing a mistake, and what does that suggest about which projects are worth doing at work?
-

Key terms

portfolio project · analysis plan · decision log · confounder · segment effect · comparison group · discount band · threshold sensitivity · effect size · statistical significance · confidence interval · null result · selective reporting · garden of forking paths · denominator of the search · FINDINGS file · README first paragraph · commit history as evidence · ninety-second scan · the two-minute version · reconciliation · job-ready

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 68, How Data Hiring Works**, is the other side of section 27.9: what the people reading your repository are being asked to decide, and when in the process they read it.
- **Chapter 69, The Extra-Points Method**, turns the two-minute version into a repeatable way of answering any interview question.
- **Chapter 82** sets take-home assignments, which are graded on exactly the standards in this chapter, and runs mock interviews on projects like this one.
- **Chapter 28** is where the SQL in section 27.2 becomes the SQL of someone who designs the database, with window functions, performance, and modeling.
- **Chapter 29** turns the script in section 27.5 into software: a package, tests, and a command-line interface.
- **Chapter 30** builds the experiment the memo proposes, which is the only honest way to answer the causal question this chapter had to decline.
- **Chapter 31** is what you reach for when the experiment is impossible, which at most companies it usually is.
- **Chapter 39** takes the honesty discipline of section 27.8 into model evaluation, where the same five moves have machine learning names.
- **Chapter 44** is Part IV's capstone: the same arc with a model in the middle, ending in a ranked call list rather than a recommendation.
- **Chapter 83, The Long Game**, is about what happens after the first job, which the portfolio exists to get.

Answers to practice exercises

- (a) Fails the first test: no decision behind it. Nothing you find can be right or wrong. (b) Fails the second: the answer is fixed before the work starts, so the project is an illustration. The fix is the word "whether" in place of "that". (c) Fails the third, and arguably the first: nobody is annoyed if it is wrong, because nobody is deciding anything with it. It is a report, not an analysis, and it belongs in a dashboard rather than a portfolio.
- `COUNT(*)` counts order *lines*, not orders, because the join to `order_items` puts an order on the table once per product on it. It would have been **higher**, by the average number of lines per order: the full dataset has 209,006 lines across 116,194 orders, so roughly 1.8 times higher. Every customer would look like a more frequent buyer than they are, and the error would be larger for Wholesale, whose orders carry more lines, which would have made the wrong headline look even stronger.
- Because averaging percentages gives every line equal weight regardless of size. A customer with one ₹200,000 crate order at 10% and nine ₹2,000 orders at 0% has an average line discount of 1%, and an actual discount rate of about 9.2%. The question is about money given away, so the calculation has to be weighted by money. This is the same reason a company-wide "average margin" is computed from summed revenue and summed cost, never from the mean of per-order margins.
- Any two of: the headline gap reproduces in 2024, so it is stable, and stability is not causality; Retail and Hospitality have too few deep-discount customers (10 and 9) to test the claim there at all, which is an absence of evidence rather than evidence of absence; the `band_cut` at 5% is a choice, and moving it to 3% shrinks the gap by more than a third; the within-Wholesale correlation is statistically significant, which would superficially support a relationship, and is far too small to matter.
- The prediction most people write is "larger, because the two groups are further apart". That is right, and the reason is not the one they have in mind:

	customers	avg_orders	avg_revenue
8% or deeper	576	15.6	484038.0
under 8%	4023	9.3	215719.0

The gap in average orders goes 3.5 at `band_cut=3`, 5.5 at 5, and 6.3 at 8. Now look at who is in the "deep" group at each setting:

<code>band_cut</code>	Deep group	Of which Wholesale	Average orders of the rest
3.0	840	662	4.5
5.0	681	662	1.2
8.0	576	576	none left

Raising the threshold does not select more heavily discounted customers. It **removes non-Wholesale customers from the deep group**, and they were the ones dragging its average down. At 8% the deep group is 100% Wholesale, and the "finding" is at its most impressive precisely when it has stopped measuring discounting altogether.

So the threshold is a dial on segment purity wearing the label of discount depth, and every value of it produces a true, quotable, larger-sounding number. That is the whole of section 27.8 in one parameter.

6. A model answer:

Claim. Deeper discounts cause customers to order more often, so raising Retail discounts would raise Retail order frequency.

Test. Compare order counts across discount depth among customers who are otherwise comparable, which means within a single segment, using 2025 and replicating on 2024. Discount rate is computed per customer as net over list revenue.

Drop it if. Within-segment comparison shows no consistent relationship between discount depth and order count, or the difference between the deepest and shallowest groups has a confidence interval that includes zero.

The value of writing that third paragraph before running anything is that the actual result is exactly the case it names, so there is no room afterward to decide that the split "was not the right comparison".

7. The confounder is **existing engagement**: customers who already buy more are more likely to open your emails, so opening is a symptom of being an active customer rather than a cause of orders. The comparison to run is a randomized one: pick a group of comparable customers, send to some and not others, and compare order rates. Failing that, at minimum compare within a band of prior-year order frequency, so that heavy and light buyers are not being compared to each other. The giveaway in the original sentence is that the proposed action ("send more emails") does not follow even if the correlation is real: sending more emails to people who do not open them changes nothing.

8. For 662 customers: *"Within Wholesale, discount depth and order frequency are essentially unrelated ($r = -0.08$). The relationship reaches statistical significance only because the sample is large, and if anything it points slightly the wrong way for the proposal."* For 40 customers with the same r : *"Within Wholesale we see no relationship between discount depth and order frequency ($r = -0.08$), but with 40 customers this analysis could not have detected a moderate effect either. It does not support the proposal and it does not rule it out."* Same statistic, two different honest sentences, because the second sample cannot distinguish "no effect" from "not enough data".

9. A model answer for a Chapter 20 automated report project:

Riverstone's sales head was spending the first forty minutes of every morning rebuilding the previous day's numbers by hand, which meant the 9 a.m. review often started without them. This repository is the script that replaced that: it reads yesterday's orders, produces the flash summary with the comparisons she actually used, and emails it at 7:30. The daily number now arrives before she does.

Under sixty words, no tool names, and the reader knows what decision it serves.

10.

What I checked

- 2025 headline: 5%+ discount customers place 14.8 orders vs 9.3. Reproduces in 2024 (13.4 vs 8.6).
- Split by segment: 662 of 681 deep-discount customers are Wholesale; no Wholesale customer is below 5%.
The headline is a segment comparison.
- Within Wholesale, by discount quartile: 14.1 / 16.7 / 16.4 / 13.6 orders. $Q4 - Q1 = -0.48$, 95% CI [-1.89, 0.92], $p = 0.50$. No relationship.
- Retail and Hospitality: 10 and 9 customers above 5%, averaging ~1 order each. Too few to test; not evidence of no effect.
- Threshold sensitivity: band_cut 3 / 5 / 8 gives gaps of 3.5 / 5.5 / 6.3 orders. The gap grows because raising the cut removes non-Wholesale customers from the deep group, not because discount depth matters.
- Duplicate customer merge (48 records, 621 orders) moved the headline by 0.2 orders. Kept the merge.

Six lines rather than four, which is the right direction to be wrong in.

11. Randomize at the customer level among Retail customers who ordered at least twice in 2025, which excludes the one-order accounts that distorted the original comparison. Roughly half get an additional standing discount of five percentage points for two quarters; the rest carry on unchanged. The primary measure is orders per customer over those two quarters; secondary measures are net revenue per customer and gross margin per customer, because the discount can win the first and lose the third. Two quarters is chosen so the comparison spans one weak season and one normal one. The policy change is justified if the treated group's order frequency is higher by enough that the extra gross profit exceeds the margin given away, which for Retail at about 30% margin means roughly a fifth more volume at a five-point discount before it breaks even. That number belongs in the plan before the test runs, because afterward it is negotiable.

12. Two explanations. **Noise:** with 166 customers per quartile and a standard deviation of several orders, a spread of three orders between quartiles is well within what chance produces, and the confidence interval on the ends already includes zero. **A real middle effect:** medium-sized Wholesale accounts might order more frequently in smaller quantities, while the largest accounts consolidate into fewer, bigger orders and negotiate the deepest discounts, which would put the top quartile low for a reason that has nothing to do with the discount. To tell them apart, look at order size by quartile rather than count, and at revenue per customer, which is flat across the four. If the deepest quartile has the largest orders and similar revenue, the second explanation is doing the work; if nothing has a pattern, the first is.

13. What changes: the recommendation is no longer about Retail growth but about the approval thresholds, so the lead becomes something like *"The 5% and 10% approval steps are working as intended in Wholesale and are not being reached in Retail or Hospitality, where only 19 customers in 2025 crossed 5% at all."* The margin paragraph moves up, because it is the finance manager's decision variable. The proposed test moves down or out; it is not their call. What stays: the segment finding, because it is the reason the thresholds look different by segment; the null result, in the same position; and the two data problems, which matter more to finance than to sales, since a missing sales rep on 3,414 orders breaks any commission or coverage report.

14. They do not believe the file on its own, and that is the right instinct. What they believe is the **combination:** a FINDINGS file whose null results are specific and checkable against the code and the commit history, which was written over weeks and cannot be reconstructed afterward. A fabricated

null result is harder to invent convincingly than a finding, because it has to be consistent with the data that is sitting right there. The deeper answer is that the file's real value is not proof, it is a prompt: it gives the interviewer something specific to ask about, and forty-five seconds of you explaining why you dropped a comparison is evidence that no file could supply.

15. Present it as a decision, not as a null result. "I was asked whether we should discount harder in Retail. The answer was no, and the analysis that showed why took four days." The value is the margin not spent, which is estimable: name it. On what it suggests about which projects are worth doing, the uncomfortable part is that a project that prevents a mistake is worth more and shows worse than one that ships a dashboard, so at work you have to make the prevention visible yourself, in writing, at the time. The habit of writing the memo, rather than sending the answer in a chat message, is most of what makes the difference between having done the work and being known to have done it.
